POPPIE ORACLE
Security Review

JUNE, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors

PeterSR

0x539.eth

Table of Contents

Protocol Summary

Custom oracle contracts for the Poppie x Euler V2 lending market (tokenized US equities as collateral, USDT borrowable).

The Oracle stores the latest price for each supported tokenized stock. The Adapter translates those prices into the format that Euler (the lending protocol) expects.

`PoppieEulerOracle` is the core contract. It stores the latest price for each asset, accepts price pushes from the keeper, and enforces all safety guards (circuit breaker, cumulative deviation cap, pause logic). This is where all state lives.

`PoppieEulerAdapter` is a thin, read-only translator. It implements Euler's ERC-7726 interface so Euler vaults can query prices in the format they expect. The adapter reads from the oracle and handles decimal-scale conversion. It has no state of its own and enforces no rules — all validation happens in the oracle.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

17f48b8124f52a68ae0186e9fc80970dc6eeffc0

Scope

Id	Files in scope
1	PoppieEulerOracle.sol
2	PoppieEulerAdapter.sol
3	IPriceOracle.sol
4	IPoppieEulerOracle.sol

Roles

Id	Roles
1	Admin
2	Keeper
3	User

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	0	Significant risks
Low	3	Minor issues with low impact
Informational	6	Best practices or suggestions
Gas	0	Optimization opportunities

Issues Statuses

Id	Title	Status
L-01	Anchor Rotation At Window Boundary Can Trap Price Above Current Level	Fixed
L-02	No Direct Unpause Mechanism — Recovery Requires Keeper Cooperation	Acknowledged
L-03	Deployer's Private Key Is Directly Loaded From Environment Variables	Acknowledged
I-01	'EIP7726' Is Not Recommended In Its Current State	Acknowledged
I-02	Token Decimals Not Verified	Fixed

Id	Title	Status
I-03	‘maxPriceAge‘ Is Global And Not Per Asset	Fixed
I-04	Adapter Base Registration Accepted Without Oracle Price Availability	Fixed
I-05	No Event Emitted On Change Of Anchor Price Or Timestamp	Fixed
I-06	‘NoPendingAdmin‘ Error Is Emitted Even When There Is A Pending Admin Set	Fixed

Security Grade

Based on the audit findings and code quality, the overall protocol security grade is 97.50 /100.00. This reflects an excellent security posture with almost no room for improvement.

Findings

Low

Low 01 Anchor Rotation At Window Boundary Can Trap Price Above Current Level

Location

PoppieEulerOracle.sol

Description

When the anchor window expires, the new anchor is set to `cfg.lastPrice` — the *previous* keeper push — before evaluating the *current* incoming push:

```
if (_window ≠ 0 && uint256(ts) - uint256(cfg.anchorTimestamp) > _window) {
    newAnchor = cfg.lastPrice;          // rotates to previous push
    cfg.anchorPrice = newAnchor;
    cfg.anchorTimestamp = ts;
}
uint256 dev = _deviationBps(newPrice, newAnchor);
if (dev > cap) revert CumulativeDeviationExceeded( ... );
```

This creates an asymmetry when prices move directionally across a window boundary.

Consider:

Step	Last Price	Anchor Price	Incoming Push	Deviation	Outcome
Mid-window	100	80	–	–	anchor = 80, within cap
Window expires	100	80	60	$\|60 - 100\|/100 = 40\%$	checked against new anchor 100
Next push	60	100	50	$\|50 - 100\|/100 = 50\%$	may exceed cap

After the rotation the anchor sits at 100 while the accepted price is 60. Any subsequent push continuing the downtrend is measured from 100 rather than from 60 — artificially inflating the

apparent deviation. A real market move of 60->50 (17%) is reported as 50% against the rotated anchor, likely triggering `CumulativeDeviationExceeded` and stalling the feed.

The effect is symmetric for uptrends (anchor trapped below current price) but the practical risk is higher on downtrends: equity price feeds drop sharply during market stress, exactly when oracle freshness matters most.

Recommendation

Rotate the anchor to the *current* incoming price rather than the previous push, so the new window always starts from the just-accepted level:

```
if (_window ≠ 0 && uint256(ts) - uint256(cfg.anchorTimestamp) > _window) {
    newAnchor = newPrice;   // anchor the new window at the price being accepted now
    cfg.anchorPrice = newAnchor;
    cfg.anchorTimestamp = ts;
}
```

This eliminates the boundary gap: every new window begins from the last accepted price, and the cumulative cap measures drift from that point forward consistently.

Status

Fixed

Low 02 No Direct Unpause Mechanism — Recovery Requires Keeper Cooperation

Location

PoppieEulerOracle.sol

Description

The `pauseAssets` function can pause assets, but the unpause mechanism is not directly controlled by the admin. Once an asset is paused, the only way to unpause it is through the `keeperPushPrices` function's auto-unpause logic, which automatically unpauses if the keeper successfully pushes a valid price that passes all guards.

This creates an operational dependency: the admin cannot directly unpause an asset. The admin's only recovery path is to call `adminSetPrice` to set a price reference, but this leaves `paused = true`. The asset only becomes unpaused when the keeper subsequently calls `keeperPushPrices` and the price passes validation.

If the keeper is unavailable, offline, compromised, or refuses to cooperate, the asset remains paused indefinitely. While the admin can rotate the keeper address, this still requires finding or deploying a new keeper and having it push a price — there is no emergency unpause mechanism controlled solely by the admin.

Proof of Concept

Consider the following scenario:

1. Pause triggered: Admin or keeper calls `pauseAssets([asset])`, setting:

```
cfg.paused = true;
cfg.lastPrice = 0;
cfg.lastPriceTimestamp = 0;
cfg.anchorPrice = 0;
cfg.anchorTimestamp = 0;
```

2. Recovery attempt: Admin calls `adminSetPrice(asset, recoveryPrice)` to set a reference price:

```
cfg.lastPrice = recoveryPrice;
cfg.lastPriceTimestamp = ts;
cfg.anchorPrice = recoveryPrice;
cfg.anchorTimestamp = ts;
// But: cfg.paused remains true
```

3. Keeper unavailable: The keeper is offline or has been compromised. It cannot or will not call `keeperPushPrices`.
4. Deadlock: Calls to `getPrice(asset)` revert with `AssetIsPaused(asset)` indefinitely. The asset cannot be unpaused without keeper cooperation.
5. Limited options:
    - Admin can rotate the keeper address via `setKeeper(newKeeper)`, but still requires the new keeper to push
    - No admin-only unpause function exists to break the deadlock

Recommendation

Add an admin-only emergency unpause function:

```solidity
/// @notice Emergency unpause for an asset. Only callable by admin.
/// @param asset The asset to unpause.
function adminUnpause(address asset) external override onlyAdmin {
    if (!_assetConfigs[asset].configured) revert AssetNotConfigured(asset);
    _assetConfigs[asset].paused = false;
    emit AssetUnpaused(asset);
}
```

This provides the admin with direct control to recover from a paused state without depending on keeper availability. Alternatively, the `pauseAssets` function could have stricter access control (e.g., only admin, not keeper), reducing the scenarios where unintended pauses occur.

Status

Acknowledged

Low 03 Deployer's Private Key Is Directly Loaded From Environment Variables

Location

DeployDev.s.sol

ConfigureDev.s.sol

Description

Both deployment scripts load the deployer's private key directly from environment variables using `vm.envUint("DEPLOYER_KEY")`. This is a security anti-pattern because environment variables can be exposed through multiple vectors:

- Process inspection: Commands like `ps aux` or `env` can reveal environment variables to other users or processes
- CI/CD logs: Build logs, GitHub Actions logs, or similar CI/CD pipeline outputs often capture environment variables, especially when debugging is enabled
- Shell history: Commands using environment variables get saved in shell history files (`.bash_history`, `.zsh_history`, etc.), which may be world-readable or backed up
- Log aggregation: Container logs, system logs, and log aggregation services may capture environment variables

- Container image inspection: Docker images or Kubernetes pod descriptions can expose environment variables
- Process monitoring: System monitoring tools or APM solutions may log environment data

While these scripts are marked as "dev" deployments, the same pattern could be adopted for production deployments, significantly increasing the attack surface. An attacker gaining access to any of these exposure vectors obtains the deployer's private key, enabling them to redeploy contracts, drain assets, or modify critical configurations.

Proof of Concept

Scenario 1: CI/CD Pipeline Exposure

1. Engineer commits `script/DeployDev.s.sol` to GitHub
2. CI/CD pipeline runs: `DEPLOYER_KEY=0x ... forge script ...`
3. Build log is temporarily visible or archived
4. Attacker gains read access to CI/CD logs (through GitHub compromise, insider, etc.)
5. Attacker extracts `DEPLOYER_KEY` from logs and signs malicious transactions

Scenario 2: Shell History

1. Engineer runs deployment manually: `DEPLOYER_KEY=0xabc123 ... forge script DeployDev.s.sol --broadcast`
2. Shell saves command to `.bash_history`
3. Attacker gains shell access (compromised account, shared machine, etc.)
4. Attacker reads shell history and extracts the private key

Scenario 3: Process Inspection

1. Deployment script is running (takes 30 seconds)
2. Attacker on same machine runs `ps aux` or inspects `/proc/[pid]/environ`
3. Environment variable `DEPLOYER_KEY` is visible
4. Attacker extracts the key

Recommendation

Use a secure secrets management system instead:

Option 1: HashiCorp Vault

```
// Read from Vault via CLI before execution
bytes32 key = vm.envBytes32("DEPLOYER_KEY_FROM_VAULT");
```

Deploy with: `DEPLOYER_KEY_FROM_VAULT=$(vault kv get -field=key secret/deployer) forge script ...`

Option 2: AWS Secrets Manager

```
DEPLOYER_KEY=$(aws secretsmanager get-secret-value --secret-id deployer-key --query
    SecretString --output text) forge script ...
```

Option 3: Hardware Wallet / Signing Service Use tools like:

- Foundry's keystore: `forge script ... --keystore ~/.foundry/keystores/deployer`
- Hardware signers: Ledger, Trezor, or Gnosis Safe integration
- AWS KMS: Sign transactions without exposing the key
- Burner accounts: Use ephemeral keys for dev, separate higher-privilege keys for production

Best Practice for Production:

- Never load private keys into smart contract scripts
- Use a separate signing service (AWS KMS, Fireblocks, Gnosis Safe)
- Require multi-sig approval for deployment
- Use timelocks for admin functions
- Rotate keys regularly and store offline (cold wallet)

Status

Acknowledged

Response: For any production deployment with meaningful admin authority the deployer should be a Foundry key store (`cast wallet import`, then `--account`), a hardware signer (`--ledger`, `--trezor`), or a KMS-backed signer rather than a raw env-var key. These dev scripts retain the env-var path for repeatability; a separate production deploy script should not.

Informational

Info 01 EIP7726 Is Not Recommended In Its Current State

Location

PoppieEulerAdapter.sol:14

Description

`PoppieEulerAdapter` implements `EIP-7726`, an interface that is explicitly marked as not recommended for use in the EIP itself. The `EIP-7726` specification notes that it should not be relied upon in production due to its experimental/draft status and the risks it poses to integrators who may assume security guarantees that the interface does not provide.

Recommendation

To mitigate follow the given steps:

1. Document the known limitation prominently in the NatSpec and README so integrators are aware they are consuming a draft/experimental interface.
2. Evaluate whether a more stable oracle interface (e.g., Chainlink `AggregatorV3Interface`, or a bespoke internal interface) would be a safer long-term choice for the protocol.
3. If `EIP-7726` conformance is required for Euler compatibility, add a `getOracleInfo()` or equivalent method that returns metadata (staleness window, price source) so callers can make informed trust decisions.

Status

Acknowledged

Info 02 Token Decimals Not Verified

Location

PoppieEulerAdapter.sol:98

Description

In `registerBase`, the admin supplies the `decimals` value for a token manually. There is no on-chain verification that this value matches the actual `decimals` return value of the ERC-20 token being registered.

```
uint256 denominatorExp = uint256(info.decimals) + MAX_DECIMALS - uint256(
    unitOfAccountDecimals);
return Math.mulDiv(inAmount, uint256(price), 10 ** denominatorExp);
```

Recommendation

Call `decimals` on the token inside `registerBase` and cross-check the caller-supplied value, or use it directly:

```
function registerBase(address base, uint8 decimals) external onlyAdmin {
    if (base == address(0)) revert ZeroAddress();
    if (base == unitOfAccount) revert BaseIsUnitOfAccount();
    if (_baseInfo[base].registered) revert BaseAlreadyRegistered(base);
    if (decimals > MAX_DECIMALS) revert DecimalsTooLarge(decimals);

    // Cross-check against the token's own decimals() to prevent silent misconfiguration
    .
    try IERC20Metadata(base).decimals() returns (uint8 onChainDecimals) {
        if (onChainDecimals ≠ decimals) revert DecimalsMismatch(decimals,
    onChainDecimals);
    } catch {} // tokens without decimals() are still allowed; admin value used as-is

    _baseInfo[base] = BaseInfo({registered: true, decimals: decimals});
    emit BaseRegistered(base, decimals);
}
```

Alternatively, read `decimals` directly and drop the admin-supplied parameter, using `18` as a fallback for tokens that do not implement it.

Status

Fixed

Info 03 `maxPriceAge` Is Global And Not Per Asset

Location

PoppieEulerOracle.sol:204

Description

`maxPriceAge` is a single global value shared across all configured assets. Every asset's freshness is judged against the same staleness window:

```
uint256 _maxAge = maxPriceAge;
if (_maxAge ≠ 0) {
    if (block.timestamp - uint256(cfg.lastPriceTimestamp) > _maxAge) revert StalePrice()
    ;
}
```

Tokenised US equities trade on different schedules and have different liquidity profiles. A pre-market equity future may legitimately go hours without a keeper push during overnight or weekend hours, while a liquid large-cap equity should be considered stale within minutes during trading hours. A single global `maxPriceAge` forces the operator to choose between:

- Too tight: a short window that causes assets with infrequent legitimate updates (off-hours, low-volume names) to revert with `StalePrice`, freezing the connected Euler vault for that collateral.
- Too loose: a long window that allows genuinely stale prices for actively-traded assets to pass through without reverting, exposing the Euler vault to mispriced collateral.

The only current workaround is `setMaxPriceAge(0)` to disable staleness globally — which removes the safety guarantee for all assets simultaneously.

Recommendation

Move `maxPriceAge` into `AssetConfig` as a per-asset field and add a setter:

```
// In AssetConfig struct:
uint40 maxPriceAge;   // 0 = staleness check disabled for this asset

// In getPrice:
uint256 _maxAge = uint256(cfg.maxPriceAge);
if (_maxAge ≠ 0 && block.timestamp - uint256(cfg.lastPriceTimestamp) > _maxAge) {
    revert StalePrice();
}

// New setter:
function setAssetMaxPriceAge(address asset, uint40 newMaxAge) external onlyAdmin {
    if (!_assetConfigs[asset].configured) revert AssetNotConfigured(asset);
    _assetConfigs[asset].maxPriceAge = newMaxAge;
    emit AssetMaxPriceAgeUpdated(asset, newMaxAge);
}
```

The global `maxPriceAge` can be retained as a deployment-time default applied during `configureAssets`, with the per-asset setter available for overrides. This allows tighter windows on liquid assets and looser windows on assets with legitimate update gaps, without compromising the staleness guarantee for either.

Status

Fixed

Info 04 Adapter Base Registration Accepted Without Oracle Price Availability

Location

PoppieEulerAdapter.sol:90-101

Description

`registerBase` succeeds independently of the master oracle's state for the same asset — it does not call `master.getPrice(base)` or `master.getAssetConfig(base)` to verify a usable price exists:

```
function registerBase(address base, uint8 decimals) external onlyAdmin {
    if (base == address(0)) revert ZeroAddress();
    if (base == unitOfAccount) revert BaseIsUnitOfAccount();
    if (_baseInfo[base].registered) revert BaseAlreadyRegistered(base);
    if (decimals > MAX_DECIMALS) revert DecimalsTooLarge(decimals);
    _baseInfo[base] = BaseInfo({registered: true, decimals: decimals});
    emit BaseRegistered(base, decimals);
}
```

If an admin registers a base whose oracle entry is either unconfigured or configured-but-unseeded, subsequent `getQuote` calls revert with `PriceNotInitialized`. The adapter advertises a registered, priceable asset that cannot actually be priced until admin completes the oracle-side `adminSetPrice` step.

Recovery is one admin call and the failure mode is functionally identical to a routine off-hours freeze.

Recommendation

At `registerBase`, probe the master oracle and revert if no usable price exists:

```
try master.getPrice(base) returns (int256) { }
catch { revert OracleNotReady(base); }
```

Alternatively, document clearly that adapter registration and oracle seeding are independent admin steps and must be coordinated in the correct order.

Status

Fixed

Info 05 No Event Emitted On Change Of Anchor Price Or Timestamp

Location

PoppieEulerOracle.sol:242

Description

When the anchor window expires in the `_checkAndUpdateAnchor` function, the contract silently updates two critical state variables — `cfg.anchorPrice` and `cfg.anchorTimestamp` — without emitting an event.

Specifically, when the condition `_window ≠ 0 && uint256(ts) - uint256(cfg.anchorTimestamp) > _window` is met (anchor window has expired), the anchor rotates to the previous keeper push (`cfg.lastPrice`), and both the anchor price and timestamp are updated. This represents a significant state transition in the oracle's cumulative deviation guard mechanism, yet produces no observable event.

This lack of event emission creates observability and auditability gaps:

- Off-chain monitoring systems cannot detect when anchor rotations occur
- Governance or risk management tools cannot track the frequency or timing of anchor resets
- Dependent protocols relying on the oracle have no transparent record of when the anchor reference point changes
- External systems cannot react to this significant state change

Recommendation

Emit an event when the anchor window expires and the anchor is rotated. Add a new event definition:

```
event AnchorRotated(address indexed asset, uint128 oldAnchor, uint128 newAnchor, uint40
    timestamp);
```

Then update `_checkAndUpdateAnchor` to emit this event at line 274:

```
if (_window ≠ 0 && uint256(ts) - uint256(cfg.anchorTimestamp) > _window) {
    uint128 oldAnchor = cfg.anchorPrice;
    newAnchor = cfg.lastPrice;
    cfg.anchorPrice = newAnchor;
    cfg.anchorTimestamp = ts;
    emit AnchorRotated(asset, oldAnchor, newAnchor, ts);  // Add this line
}
```

This provides full transparency for off-chain systems and improves auditability of the oracle's behavior.

Status

Fixed

Info 06 `NoPendingAdmin` Error Is Emitted Even When There Is A Pending Admin Set

Location

PoppieEulerOracle.sol

PoppieEulerAdapter.sol

Description

The `acceptAdmin` function in both contracts uses the error name `NoPendingAdmin` when the caller is not the pending admin, but this error name is semantically incorrect. The error `NoPendingAdmin` implies "there is no pending admin set," when the actual condition being checked is "the caller is not authorized as the pending admin."

This creates ambiguity in error handling: callers cannot distinguish between two different failure scenarios:

1. No pending admin exists (legitimate state where `pendingAdmin == address(0)`)
2. A pending admin exists, but the wrong account is calling (security scenario where an unauthorized address attempts to claim admin rights)

Both scenarios revert with the same `NoPendingAdmin` error, making it impossible for off-chain monitoring, tests, or error handling logic to differentiate between a configuration issue and an unauthorized access attempt.

Proof of Concept

Consider the following sequence:

1. Admin calls `transferAdmin(address newPendingAdmin)` -> sets `pendingAdmin = 0xBob`
2. Address `0xAlice` (unauthorized) calls `acceptAdmin`
3. Transaction reverts with error `NoPendingAdmin`
4. Off-chain monitor/client receives error `NoPendingAdmin` and cannot tell if:
    - No pending admin was ever set (benign state), or
    - An unauthorized account tried to hijack admin rights (security event)

Recommendation

Consider adding a check to emit a more specific error when truly no pending admin has been set:

```
function acceptAdmin() external override {
    if (pendingAdmin == address(0)) revert NoPendingAdmin();
    if (msg.sender ≠ pendingAdmin) revert NotPendingAdmin();
    emit AdminTransferred(admin, msg.sender);
    admin = msg.sender;
    pendingAdmin = address(0);
}
```

This distinguishes between:

- `NoPendingAdmin`: No pending admin was ever set (configuration state)
- `NotPendingAdmin`: Caller is not the authorized pending admin (authorization failure)

Status

Fixed